Eric Roberts and Jed Rembold
CS 151

Section #2
Week of January 30, 2023

Simple Python

1. Precedence

Evaluate the following expressions using Python's rules of precedence:

1a) `3 + 3 * 3 * 3 + 3 + 3 * 3`

1b) `10 ** ((9 + 8) % 7) + 6 * (5 + 4 + 3 // 2 + 1)`

2. Functions

Write and test a Python function `max3(a, b, c)` that returns the largest of its three arguments but does so without calling any built-in Python functions. Later in the semester, you will learn how to write a version of `max` that takes any number of arguments, as the library version of this function does.

3. Hailstone sequence

As far as I know, the only computer scientist to win a Pulitzer Prize is Douglas Hofstadter, who won one for *Gödel, Escher, Bach,* a delightful exploration of the intricacies of computation. Hofstadter's book includes many famous mathematical puzzles, including this one:

> Pick some positive integer and call it n.
> If n is even, divide it by two.
> If n is odd, multiply it by three and add one.
> Continue this process until n is equal to one.

On page 401 of the Vintage edition, Hofstadter illustrates this process with the following example, starting with the number 15:

15	is odd, so I make $3n+1$:	46
46	is even, so I take half:	23
23	is odd, so I make $3n+1$:	70
70	is even, so I take half:	35
35	is odd, so I make $3n+1$:	106
106	is even, so I take half:	53
53	is odd, so I make $3n+1$:	160
160	is even, so I take half:	80
80	is even, so I take half:	40
40	is even, so I take half:	20
20	is even, so I take half:	10
10	is even, so I take half:	5
5	is odd, so I make $3n+1$:	16
16	is even, so I take half:	8
8	is even, so I take half:	4
4	is even, so I take half:	2
2	is even, so I take half:	1

As you can see from this example, the numbers go up and down, but eventually—at least for all numbers that have ever been tried—comes down to end in 1. In some respects, this

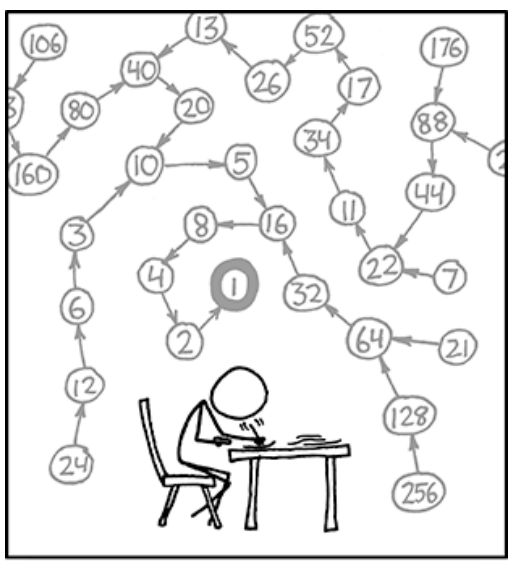
process is reminiscent of the formation of hailstones, which get carried upward by the winds over and over again before they finally descend to the ground. Because of this analogy, this sequence of numbers is usually called the ***Hailstone sequence,*** although it goes by many other names as well.

Write a function `hailstone` that takes an integer and then displays the Hailstone sequence for that number, just as in Hofstadter's book, followed by a line showing the number of steps taken to reach 1. For example, your program should be able to reproduce the following IDLE session:

IDLE

```
>>> from Hailstone import hailstone
>>> hailstone(17)
17 is odd, so I make 3n+1: 52
52 is even, so I take half: 26
26 is even, so I take half: 1
13 is odd, so I make 3n+1: 40
40 is even, so I take half: 20
20 is even, so I take half: 10
10 is even, so I take half: 5
5 is odd, so I make 3n+1: 16
16 is even, so I take half: 8
8 is even, so I take half: 4
4 is even, so I take half: 2
2 is even, so I take half: 1
The process took 12 steps to reach 1.
>>>
```

The fascinating thing about this problem is that no one has yet been able to prove that it always stops. The conjecture that this process always terminates is called the ***Collatz conjecture,*** and appears in the following *XKCD* cartoon by Randall Munroe:



THE COLLATZ CONJECTURE STATES THAT IF YOU PICK A NUMBER, AND IF IT'S EVEN DIVIDE IT BY TWO AND IF IT'S ODD MULTIPLY IT BY THREE AND ADD ONE, AND YOU REPEAT THIS PROCEDURE LONG ENOUGH, EVENTUALLY YOUR FRIENDS WILL STOP CALLING TO SEE IF YOU WANT TO HANG OUT.